

BuildWise: Consolidated SE3090 Report Draft

Group: G07 (existing package identifier; confirm before submission)

Revision date: 2026-10-05. Repository: <https://github.com/IT24102414/build>

Status: evidence-backed technical draft. It is not a signed final submission. The specification deadline was 2026-09-30; this revision does not assert an earlier submission. Individual reflections, signatures, deployment links and video require student input.

Part A: Group report

1. Problem, objectives and scope

Construction projects need a traceable connection between requested materials, supplier offers, purchasing decisions, receiving records and quality outcomes. BuildWise records that lifecycle in one shared system. Operational staff work on mobile; desk users review data and decisions on web. The same operations are available to authorized roles in Flutter so cross-platform access does not change business rules.

The four primary business components are material request/approval, supplier/RFQ/quotation/procurement, delivery/receiving, and quality/NCR management. Suppliers are external contacts approached by email, not authenticated internal users. The seven roles and demo accounts are listed in the root README.

2. Functional requirements and rules

Material requests contain project, material, unit, quantity, priority, reason and dates. Typed material names accept letters and numbers and may use suggestions. Site Engineers create their requests. Site Officers track their own request records and receive deliveries. Approvers review demand; the procurement officer records supplier offers and initiates analysis; the procurement manager records the purchasing decision.

Approval creates a confirmed purchase order through the API transaction. The receiving list refreshes to make that order available. Recording deliveries maintains ordered/received/damaged/shortage context. Inspection materials must belong to the selected delivery and inspected quantities cannot exceed the corresponding received quantities.

Inspection rules: positive inspected quantity; non-negative rejected quantity; rejected no larger than inspected; accepted plus rejected equals inspected; at least one line; notes for failed checklist items; reason for rejected quantities; at most ten evidence files, each at most 10 MB. Rejected quantity zero creates no NCR. A positive rejection automatically creates an NCR. Resolved, Closed and AcceptedException require a resolution. Backend transition validation remains authoritative.

Role navigation follows the React navigation table. Flutter Site Officers use the own-request endpoint rather than the full register. Field/quality users can read purchase orders with commercial values redacted by the API. Camera/gallery evidence and local notifications are device capabilities; hardware permission behavior still requires an installed-device run.

3. Integrated architecture

Public clients -> ASP.NET Core API -> PostgreSQL and internal Python agent services.

React and Flutter call only the ASP.NET API. Authentication uses hashed passwords and JWTs; authorization policies protect operations. EF Core coordinates relational persistence. Controllers accept DTOs, services enforce rules, and middleware handles auditing/error concerns. Python service ports are internal deployment endpoints, not public client integration points.

Layer	Responsibility	Evidence
React	Role routing, forms, dashboards, approval, workflow summaries	web/buildwise-web/src
Flutter	Operational forms, role workspaces, secure session, evidence capture	mobile/buildwise_mobile/lib
API	Identity, business rules, transactions, audits, workflow orchestration	backend/BuildWise.Api
PostgreSQL	Domain entities, relationships, durable workflow state	EF migrations and configurations
Agents	Domain analysis, structured outputs, bounded tools, failure results	backend/agent_service

4. Database design

Projects relate to requests; request lines relate to materials; supplier offers relate to approved requests and quotations; approved procurement produces orders and order items; deliveries relate to orders; inspections relate to deliveries and materials; rejection produces NCR follow-up. Agent workflows relate to their steps, validation outcomes and human approvals. User/role relationships govern access.

Use the EF model and migrations as the authoritative schema. The existing ERD DBML/PDF are supporting project diagrams and should be checked against the final migration set before submission. The evidence URL storage migration supports validated inline image data as well as permitted URLs. Entity DTOs expose only the information the caller's policy allows; commercial terms are redacted for field/quality roles.

PostgreSQL stores execution summaries needed for auditability, not hidden model reasoning. High-impact approval is transactional: the recorded decision and resulting confirmed purchase order must stay consistent. State fields and related-record references let the second client observe the updated outcome.

5. API design and component evidence

Component	Representative endpoints / operations	Validation and role boundary
Requests	GET/POST /api/material-requests; GET /api/material-requests/{id}; POST approval; POST request analysis	Required project/material/quantity; approver-only decisions
Procurement	Suppliers CRUD; RFQ issue/close/email; quotation create/delete/compare; workflow start/latest/decision	Officer entry/analysis; manager approval; eligibility and budget checks
Receiving	GET/POST /api/deliveries; confirmed purchase order read; delivery detail; discrepancy analysis	Authorized receiving; ordered/received/discrepancy business rules
Quality	GET/POST /api/quality-inspections; inspection risk analysis; NCR read/status operations	Delivery/material/quantity/evidence rules; valid resolution transitions

Swagger is the authoritative route/DTO reference at /swagger. The API supplies appropriate success/failure status codes and structured validation errors; clients show field/API errors rather than hiding every failure behind a generic 400.

6. React design

Functional components, hooks, React Router and Context authentication provide routing/session state. Domain service modules centralize API calls. Reusable cards, fields, tables, status badges and

error/loading/empty states serve the business pages. Role-filtered navigation and per-operation capabilities complement server enforcement.

The latest update removes unauthorized supplier prefetch when read-only users open purchase orders. Procurement workspaces load the latest persisted recommendation, making officer and manager sessions observe the same result. Responsive layout and phone-width tables were checked with local Playwright.

7. Flutter design

Flutter uses reusable widgets, StatefulWidget state, injected services for tests and secure token storage. The responsive navy menu, role labels, order and Plus Jakarta Sans typography align with React. Native operational capabilities remain camera/gallery evidence and local notifications.

Request lists refresh; procurement comparison preserves the selected request and reads the latest workflow; officers record/remove eligible offers and run analysis; managers read recommendations, edit allowed budgets and make decisions. Budget values must be non-negative or blank to clear allocation. Dashboard activity refreshes every 15 seconds while resumed and on resume. Public endpoint/status details are available without recording request bodies or secrets.

8. Agentic AI architecture and acceptance workflow

The assessed procurement path receives a domain objective, produces a structured plan and delegates tasks, reads allowed domain tools, ranks eligible offers, applies deterministic validation and pauses before purchase order creation. Authorized approval then produces an auditable result; failed validation or service failure is recorded safely.

Responsibility	Contract / controls	Visible outcome
Planning	Structured procurement objective and read-tool inputs	Plan and delegated steps
Quotation analysis	Active suppliers, offered/requested quantities and totals	Eligible offer ranking and recommendation
Deterministic validation	Schema/business rules and budget context	Validity findings and warnings
Request analysis	Request/project/items context	Planning/urgency flags
Delivery discrepancy	Ordered, received and damaged quantities	Shortage/damage findings
Quality risk	Inspection quantities/reasons	Risk result and NCR recommendation

Agents have different inputs, outputs and responsibilities. The deployed runtime may use optional LLM rationale; deterministic checks and failure behavior remain necessary when provider credentials are absent or invalid. A live golden-case walkthrough must show persisted steps, validated tool results, authorized approval and final PO, not merely a single prompt response.

9. Security and external integration

Secrets belong in environment variables or managed secret storage. Never include .env, provider keys, virtual environments, runtime dumps or request tokens in a submission archive. The updated packager uses git archive of a named committed revision rather than copying local folders indiscriminately.

Self-service API registration permits front-line roles; manager/administrator accounts are provisioned by an administrator. A separate Flutter registration interface was not evidenced in this maintenance run and remains a requirement check. SMTP access is backend-owned. Failed RFQ email is presented as failed

with retry controls; a successful HTTP request alone is not evidence that email was sent. No SMTP provider was configured in this local verification.

10. Software testing report

Suite	Observed result	Scope
.NET/xUnit	307 passed	Services, validation, authorization, controllers and API integration
Python/pytest	28 passed	Four domain agents and deterministic contract assertions
React/Vitest	163 passed	Components, forms, API errors and role behavior
Flutter	149 passed	Widgets, services, quantities, evidence, role scopes and navigation
Final Flutter auth/shell follow-up	10 passed	Required credentials, login transitions, responsive menu/actions
Production builds	React and Flutter web passed	Bundling/compilation evidence

An initial sandboxed .NET run failed on Windows EventLog permissions; the authorized rerun passed. React lint exits successfully but has existing warnings, which must not be described as zero warnings. Browser checks are read-only. Automated fixtures and widget tests do not establish every physical-device or cross-platform write operation.

A complete seven-role browser review made 179 checks and found a React supplier-prefetch permission defect. After fixing it, a targeted seven-role follow-up made 44 checks with no captured errors or page-width overflow. No unsupported Flutter role workspace or missing expected menu was reported.

11. Agent evaluation report

Evaluation combines Python contract tests with API validation and approval tests. Review `test_quotation_agent.py`, `test_request_agent.py`, `test_delivery_agent.py`, `test_quality_agent.py`, `FullLifecycleScenarioTests.cs` and RBAC tests. These exercise domain outputs and boundaries rather than relying solely on an LLM judge.

Before final demonstration, capture a golden procurement case, an ineligible supplier case, invalid quantities, missing revision comments, unauthorized approval, agent unavailability and prompt/tool-input rejection. Record workflow IDs, role, expected result, observed state and test version. Do not claim live provider behavior that was not exercised.

12. Performance report

The performance harness is `scripts/performance-test.mjs`. It measures concurrent API/database reads and each internal agent separately, recording success rates, p50/p95/max latency and throughput. The local output is `docs/performance-results-2026-10-05.json` when the run completes.

These are local Windows results, not a production SLA, network benchmark or separately isolated database latency measurement. Report concurrency, request count, timestamp and environment with the data. Repeat against deployment if production performance is claimed.

13. Deployment and CI report

GitHub Actions builds/tests .NET, Python, React and Flutter and uploads build/APK artifacts. Flutter is pinned to the verified 3.47.5 toolchain. The web API base URL is configurable using the VITE_API_BASE_URL repository variable. The deployment ADR records a proposal, not a verified public service. No successful Actions result should be asserted until the published run is inspected.

Required evaluator evidence: public React URL, API health/Swagger URLs, PostgreSQL host evidence, internal agent access from the API, current installed APK and accessible ten-minute video. The root BuildWise.apk is historical and is excluded from new source archives; supply a current verified APK to the package script. Keep access until at least 2026-10-21.

14. Demonstration sequence

1. Site Engineer creates a validated request in Flutter.
2. Site Manager opens it in React, reviews analysis and approves demand.
3. Procurement Officer records quotations and starts planning/analysis.
4. Show persisted plan, delegated steps, tool results and validation in the workflow history.
5. Procurement Manager reviews, demonstrates a missing revision-comment rejection, then approves a suitable proposal.
6. Show the confirmed purchase order and returned Flutter status.
7. Site Officer records receiving; show discrepancy analysis.
8. Quality Inspector records rejection with notes, reason and evidence; show automatic NCR.
9. Authorized manager resolves the NCR with a resolution; show rejected invalid transitions.
10. Show tests, actual Git/CI evidence and deployment links. Each member explains their component without external AI assistance.

15. Architecture decision records

The report renderer includes the ADR source documents as appendices: durable agent state, orchestration, React state, Flutter state, hosting proposal and external supplier boundaries. These historical decisions include current scope/deployment notes. Students must confirm they understand and own the decisions before signing.

16. Group AI disclosure

This maintenance and report draft were AI-assisted using Codex on 2026-10-05. Outputs and actual verification are recorded in docs/reports/2026-10-05-codex-maintenance-log.md. Existing student AI logs/declarations are historical material that the students must review and update; this report does not certify their completeness.

Part B: Individual report sections

Student 1: Peiris DPSS - Material Request and Approval

Student ID: NOT CONFIRMED. Contribution statement, owned backend/database/React/Flutter changes, distinct agent contribution, real commit/PR/test links, challenges, learning and dated AI log: TO BE COMPLETED BY THE STUDENT.

Approximately one-page reflection: STUDENT-AUTHORED TEXT REQUIRED. Signed declaration/date: NOT PROVIDED.

Student 2: Theebika - IT24102414 - Procurement

Confirm full registered name. Component scope: suppliers, RFQs, quotation comparison, workflow decisions and purchase orders. Personal contribution statement, commit/PR/test links, challenges, learning and verified AI chronology: TO BE COMPLETED BY THE STUDENT.

Approximately one-page reflection: STUDENT-AUTHORED TEXT REQUIRED. Signed declaration/date: NOT PROVIDED.

Student 3: Ramya - IT24102513 - Delivery and Receiving

Confirm full registered name. Personal contribution statement, owned cross-stack work, distinct agent contribution, commit/PR/test links, challenges, learning and verified AI log: TO BE COMPLETED BY THE STUDENT.

Approximately one-page reflection: STUDENT-AUTHORED TEXT REQUIRED. Signed declaration/date: NOT PROVIDED.

Student 4: Anoja - Quality Inspection and NCR

Student ID: NOT CONFIRMED. Personal contribution statement, owned cross-stack work, distinct agent contribution, commit/PR/test links, challenges, learning and verified AI log: TO BE COMPLETED BY THE STUDENT.

Approximately one-page reflection: STUDENT-AUTHORED TEXT REQUIRED. Signed declaration/date: NOT PROVIDED.

Part C: Submission links and references

Repository: <https://github.com/IT24102414/build>

React deployment: NOT PROVIDED. API/health/Swagger deployment: NOT PROVIDED. PostgreSQL deployment evidence: NOT PROVIDED. Current APK installation evidence: NOT PROVIDED. Ten-minute video: NOT PROVIDED.

Primary requirements source: user-supplied SE3090 Assignment 1 specification, Year 3 Semester 1, 2026. Implementation references: root README, current source, EF migrations, test suites, docs/submission-readiness.md and the listed ADRs. Library dependencies and font licensing are recorded in package manifests and assets/fonts/OFL.txt. This draft supplies no invented signatures, contribution history, performance figures or deployment results.

Observed local performance (2026-10-05)

30 requests per scenario, concurrency 5. All scenarios returned 100% successful responses.

Scenario	p95 ms	Target ms	Outcome
Shared ASP.NET Core API + PostgreSQL	78.89	2000	Passed
Agent 8001 quotation analysis	6782.05	1000	Latency target missed
Agent 8002 request analysis	94.49	1000	Passed
Agent 8003 discrepancy analysis	82.03	1000	Passed
Agent 8004 quality-risk analysis	58.02	1000	Passed

Quotation analysis missed the 1000 ms local target; investigate provider/rationale latency before claiming performance acceptance. No threshold was relaxed. API/database latency is measured together, not as an isolated database benchmark.

ADR appendix: 0001-quotation-analysis-agent-and-workflow-state.md

ADR-001 — Quotation Analysis Agent runtime and procurement workflow state

Status: Accepted · **Date:** 2026-09-19 · **Owner:** Component 2 (Supplier, Quotation & Procurement Management), SLIIT IT24102414

Scope: spec §4 (supplier selection), §5 (deterministic rules), §10 (agent) · **Evidence:** commits abdc6d5, 38bbca9, 6161ecc on feature/supplier-procurement

Context

Component 2 must turn an **approved** material request into an auditable supplier recommendation that a Procurement Manager approves **before** any purchase order exists. Three forces shaped the design:

1. **Determinism is a hard requirement.** Spec §5 lists ten business rules that ASP.NET Core must enforce as a gate; the agent may only recommend.
2. **A real agent layer is required** (SE3090): planning, tool use, structured JSON output, observable structured summaries only — no chain-of-thought.
3. **The workflow pauses on a human.** Between "recommendation produced" and "manager decides" the process idles for an unbounded time while other requests keep being processed, and every step must be reconstructable for audit.

Options considered

#	Option	Verdict
A	ASP.NET Core calls an LLM directly; the LLM picks the winner	Rejected — a non-deterministic, networked component on the critical path, untestable without provider mocking, and it would let a model decide an award.
B	Rule-based filter/rank only in C# (no agent service)	Rejected as the primary path (fails the agentic-AI requirement) but kept as the degraded path when the agent service is unreachable.
C	Python FastAPI agent microservice exposing allow-listed deterministic tools + optional LLM rationale; the C# API re-validates everything	Chosen.
D	LangGraph-style multi-agent orchestration framework	Rejected — a two-tool decision with a human-in-the-loop pause does not need a graph runtime or a checkpoint; here the pause is a database row, not a graph checkpoint.

Decision

1. **Runtime.** The agent runs as its own Python FastAPI service (backend/agent_service/quotation_agent.py), addressed by AgentService:Ur1 (<http://127.0.0.1:8001>) through /health and /analyze, called by QuotationAgentClient under a 10-second cancellation budget.

2. **Tools are allow-listed and role-separated.** The Python quotation agent exposes only `{filter_eligible, rank_by_total}`. The distinct C# `ProcurementPlanningAgent` exposes only four read tools: `GetMaterialRequest`, `GetMaterialDetails`, `GetProjectDetails`, and `GetAvailableQuotations`. It cannot approve procurement, issue a purchase order, or modify supplier records.
3. **Planning is a structured persisted contract.** `ProcurementPlanningAgentService` grounds the objective and input facts in PostgreSQL, produces ordered delegated steps, required deterministic checks and risk flags, and records summaries for all four allow-listed tools in `agent_workflow_steps.structured_result`. Only observable summaries—not hidden reasoning—are stored.
4. **The winner is decided deterministically.** Eligible = Active supplier **and** unexpired quotation. Full-coverage quotations rank first by `total_amount` ascending; a partially covering quotation can never be awarded as the sole winner — it is surfaced as a warning during comparison/enforcement.
5. **The LLM explains; it does not decide.** With `ANTHROPIC_API_KEY` set, Claude rewrites only the rationale sentence (2–3 sentences, sanitized, ≤600 chars); any failure falls back to the deterministic template rationale. A default checkout runs with **no API key and no external call**.
6. **C# never trusts the agent.** `ProcurementValidationAgent` independently checks positive quantities, Active supplier, unexpired quotation, real quotation/supplier identity, material coverage, total reconciliation, promised-delivery validity, required structured fields, and manager approval before PO authorization. Its persisted output is `{ valid, errors, warnings }`; failures move the workflow to `RevisionRequired`, never `AwaitingApproval`.
7. **Failure is contained, never silent.** Technical exceptions → workflow `Failed`; unsafe or invalid business output → `RevisionRequired` with auditable validation results. In both cases no purchase order can be created.

Workflow state persistence (PostgreSQL)

Table	Role in the workflow
<code>agent_workflows</code>	One row per run: <code>material_request_id</code> , <code>initiated_by_user_id</code> , <code>objective</code> , <code>status</code> , <code>approval_status</code> , <code>final_outcome</code> , <code>started_at</code> / <code>completed_at</code> . Survives process restarts.
<code>agent_workflow_steps</code>	Append-only, ordered audit trail: <code>agent_role</code> , <code>step_name</code> , <code>step_order</code> , <code>status</code> , <code>structured_result</code> (jsonb), <code>validation_result</code> (jsonb), <code>error_message</code> , <code>timestamps</code> .
<code>agent_approvals</code>	Append-only human decision: <code>reviewed_by_user_id</code> , <code>decision</code> (Approve / Reject / RevisionRequested), <code>comment</code> , <code>decision_date</code> .

- **Why the database, not memory.** The workflow is paused for an indefinite human window and must be re-readable later through `GET /api/procurement-workflow/{id}/history`; an in-memory queue or background scheduler would lose both the paused state and the audit trail.
- **Step isolation.** Planning (`ProcurementPlanningAgent`), analysis (`QuotationSupplierAnalysisAgent`) and validation (`ProcurementValidationAgent`) are separate rows moving `Running` → `Completed/Failed`, so a failure is attributable to a specific step.
- **Status model.** `Pending` → `Running` → `AwaitingApproval` → `Completed` | `Failed`; `RevisionRequested` returns the workflow to `Pending` for another pass instead of creating a PO.
- **Atomicity.** Purchase-order creation validates preconditions, reads the winning recommendation from the step's `structured_result`, then writes `purchase_orders` + `purchase_order_items` and repoints

quotation statuses (winner Selected, losers Rejected) inside one explicit transaction whenever the provider is relational (the in-memory test provider supports no transactions).

- **Idempotency and the approval gate.** `ValidatePurchaseOrderCreationAsync` blocks a second non-cancelled PO for the same request (§5.10) and blocks creation when no Approved `agent_approvals` row exists (§5.8).

- **Schema ownership.** EF Core migrations stay the single source of truth; the jsonb columns are mapped to string properties on the entities.

Consequences

Positive — decisions remain deterministic and unit-testable (15 xUnit tests, including a full replay of the cement scenario); an agent outage degrades instead of blocking; LLM usage is one short sentence per run, or zero by default; per-step audit trail; the cross-language JSON boundary is validated in exactly one place.

Negative / accepted — two runtimes to launch locally (mitigated by the in-process fallback and a dedicated CI job for `agent_service`); the `snake_case` ↔ `PascalCase` boundary did bite once, because `AgentRecommendationDto` is `PascalCase` while the agent emits `snake_case`, so every field bound to null (fixed in 6161ecc with `JsonNamingPolicy.SnakeCaseLower` scoped to that read); rationale text is model-generated, therefore sanitized and never used for decisions; `agent_workflows` / `agent_workflow_steps` / `agent_approvals` are shared with the other components, so column changes need team agreement.

References

- `docs/component2_spec.md` §5.7 (schema validation), §5.8 (approval gate), §5.10 (idempotency), §10 (agent contract)

- Tests: `ProcurementValidationServiceTests` (8), `ScenarioReplayTests` (1), `AuthServiceTests` (6) → 15 xUnit; `backend/agent_service/test_quotation_agent.py` (9); React vitest (15)

- Agent contract tests that caught the two agent defects:

```
test_invalid_quotation_is_not_ranked_or_recommended,
test_fallback_does_not_resurrect_ineligible_suppliers
```

ADR appendix: ADR-002-component2-agent-architecture.md

ADR-002 — Component 2: Agentic Quotation Analysis Architecture

Status	Accepted
Date	2026-09-19
Deciders	IT24102414 (Supplier, Quotation & Procurement Management)
Scope	Component 2 — Supplier, Quotation & Procurement Management
Related	docs/component2_spec.md §4.5, §5, §7, §10, §11 · commits 38bbca9, 6161ecc
Supersedes	n/a *(first ADR recorded for this component — no ADR-001 exists in docs/)*

1. Context & problem statement

SE3090 requires an Agentic AI contribution to procurement: analysing supplier quotations and recommending an award. The tension is that procurement decisions are commercially sensitive while the AI is advisory. Two failure modes had to be designed out:

1. **The AI overriding deterministic business rules** — e.g. recommending a suspended supplier because it is cheapest.
2. **Silent, unauditible AI output** — a recommendation that reaches a purchase order without an evidence trail or a human decision.

The quotation set must also be re-evaluated against live data: a supplier can be suspended *after* the AI has run, so the AI's answer can be stale.

2. Decision

D1 — A separate Python FastAPI microservice for the analysis agent.

backend/agent_service/quotation_agent.py (FastAPI, uvicorn on 127.0.0.1:8001) exposes GET /health and POST /analyze. Python was chosen over extending the C# API because the agentic layer is exploratory by nature (tool selection, prompting, LLM rationale) and is expected to iterate independently of the stable, rule-governed API. It also keeps AI dependencies out of the .NET deployable, and lets the agent be scaled, restarted or replaced without touching procurement logic.

D2 — C# ASP.NET Core remains the single source of truth.

ProcurementValidationService.cs independently re-derives eligibility (§5.1–5.10)

and re-validates the agent's answer. The agent is advisory; it cannot write to `purchase_orders`, cannot approve anything, and cannot advance a workflow past `AwaitingApproval`. The schema gate (§5.7) rejects any payload that does not match the §10 contract before it is stored.

D3 — Two-phase HTTP with an in-process fallback.

`QuotationAgentClient` calls the agent with a 10 s budget (`cts.CancelAfter(10s)`, `HttpClient` timeout 12 s). On non-success or unreachability it executes `ExecuteFallbackAnalysis`, an in-process mirror of the same deterministic rules, so procurement continues when the microservice is down.

D4 — Workflow state persisted in PostgreSQL, shared table structure.

Execution state lives in `agent_workflows`, `agent_workflow_steps` and `agent_approvals` — deliberately **shared** across all four components' agents per the ERD change-control note. Three ordered steps are recorded: `ProcurementPlanningAgent` → `QuotationSupplierAnalysisAgent` → `ProcurementValidationAgent`. Each step keeps `structured_result` / `validation_result` (jsonb), `status` and `error_message`. Only observable structured summaries are stored — no raw model chain-of-thought. Status transitions are `Pending` → `Running` → `AwaitingApproval` → `Completed` | `Failed`.

D5 — Human-in-the-loop approval gate.

`POST /api/procurement-workflow/{id}/decision` (Manager/Administrator only) records an `agent_approvals` row. Rule §5.8 blocks purchase-order creation until an `Approved` row exists; §5.10 blocks duplicate POs; §5.9 locks the source quotation and request once the PO exists.

3. Alternatives considered

Alternative	Why rejected
Agent logic inside the C# API	Couples AI iteration cadence to the stable API; agent cannot be run or scaled independently
LLM with tool-use autonomy (agent decides and acts)	Violates §10 "what it never does"; untestable against §5; risks selecting a suspended/expired supplier
Storing full model output / chain-of-thought	Unnecessary, non-auditable, prompt-injection surface
Fire-and-forget agent call	No evidence trail; §7/§11 require step-by-step history
Using the in-process fallback as the only implementation	Loses the independent cross-check that the C# and Python layers agree

4. Consequences

Positive

- Two independent implementations of the same rules cross-check each other; a divergence surfaces as a validation failure instead of a bad purchase order.
- Every recommendation is reproducible and auditable from `agent_workflow_steps`.
- Zero silent purchase orders: \$5.8/\$5.10 make a bad path fail loudly.
- The agent is independently testable (pytest) and deployable.

Negative / accepted risks

- A second runtime (Python 3.12, uvicorn) must be running; a dead agent silently degrades to the fallback rather than erroring.
- **Contract drift is the main risk class.** The C#→Python request uses `snake_case` property names; the Python→C# response originally deserialized into a PascalCase DTO with no mapping, silently producing an all-null recommendation that then failed schema validation. Fixed in 6161ecc by reading the response with `JsonNamingPolicy.SnakeCaseLower`, scoped to that read only.
- `Program.cs` originally defaulted `AgentService:Url` to port **8000** while `appsettings.json` set **8001** — a mismatch that would have silently skipped the agent and used the in-process fallback. Now aligned on **8001** in both.
- Notification e-mails are best-effort and wrapped so a failure never rolls back the transaction they report on.

5. Verification evidence

Check	Result
C# tests (dotnet test backend/BuildWise.Api.Tests)	20 passed, 0 failed — includes agent-boundary schema tests
Python agent tests (pytest test_quotation_agent.py)	9 passed, 0 failed
React tests (npx vitest run)	4 files, 15 passed
npm run build	exit 0, 58 modules
End-to-end replay (spec §11, request #1)	workflow #51 → AwaitingApproval → Approved → purchase_orders row (250 bags @ 2 100.00 = 525 000.00); quotations #3 Selected, #6/#7 Rejected

CI (`.github/workflows/ci.yml`) runs the backend, agent, web and mobile suites on push/PR to main.

6. Known gaps

- An HTTP 400 was observed on the PO-creation call that nonetheless persisted a valid PO. Not reproducible on a clean run (the identical request returned 200 OK with the correct payload). Suspected EF Core tracking conflict when `CreateFromWorkflow` re-enters `GetById` on the same scoped `DbContext`. Open, non-blocking.
- `component2_spec.md` §10's response schema omits `recommended_supplier_name`, which the implementation (and the React UI) do include.
- A quotation marked `valid = false` was previously ranked and could win; fixed in 38bbca9 (skip + strict fallback screening), with regression tests.

ADR appendix: ADR-003-react-state-management.md

ADR-003 — React State Management Strategy

Status: Accepted · **Date:** 2026-09-23 · **Owner:** Component 2 (IT24102414), agreed by all component owners

Scope: SE3090 spec §7 (React), §9 (cross-platform workflow)

Context

BuildWise's React front-end serves five role-based dashboards (Admin, Procurement Manager, Site Engineer, Delivery Staff, Quality Inspector). Each feature area has independent data lifecycles:

- **Component 1** — material requests, approval queues
- **Component 2** — suppliers, quotations, RFQs, procurement workflows, agent analysis
- **Component 3** — deliveries, receiving records
- **Component 4** — quality inspections, non-conformances

The spec requires: routing with protected routes by role, client-side validation, API error handling, loading/empty states, and testability without a running backend.

Three cross-cutting concerns shaped the choice:

1. **Auth context** — JWT, role, and logout must be available everywhere without prop drilling.
2. **Feature isolation** — each component team owns its feature folder; state solutions must not create hard cross-team coupling.
3. **Test simplicity** — vitest + React Testing Library must be able to test components without complex store setup.

Options considered

#	Option	Verdict
A	React Context + useState/useReducer per feature	Chosen — enough for SE3090 scale, testable, no extra dependencies
B	Redux Toolkit	Rejected — significant boilerplate; adds a required peer dependency; overkill for assignment scale
C	Zustand	Rejected — not a React built-in; adds a dependency; team unfamiliar
D	React Query / TanStack Query	Rejected — good for server state caching but adds complexity beyond spec requirements
E	Single global Context for all state	Rejected — causes unnecessary re-renders across all four feature areas on any state change

Decision

1. **Auth is a single global AuthContext** (`src/auth/AuthContext.jsx`) providing `user`, `token`, `login`, `logout`. All protected routes read from this context. No auth state is duplicated in feature components.
2. **RBAC guard** is a separate `accessControl.js` module with a `canAccess(role, resource)` function, keeping permission logic testable independently of components.
3. **Feature-local state** uses `useState` and `useReducer` within feature components. No cross-feature shared state exists — components communicate through the API, not through shared client-side state.
4. **API calls** use thin fetch wrappers in `src/services/*.js` (e.g., `procurementApi.js`, `qualityApi.js`). Components call these services in `useEffect`; `loading/error/data` are local `useState` variables.
5. **No external state library** is added. The single `react-router-dom` dependency handles routing; all other state is native React.

Workflow state (read-only in React)

The agentic AI workflow state (`agent_workflows`, `agent_workflow_steps`, `agent_approvals`) is **server-owned and server-persisted**. React only reads it via GET endpoints and displays it. No client-side caching of workflow state was needed.

Consequences

Positive

- Zero extra state library dependencies — consistent with the existing `package.json`
- Every component is independently unit-testable with a simple mock `AuthContext` wrapper
- Feature isolation: Component 2's procurement state does not affect Component 4's quality state
- Vitest runs pass with simple `render(<Component />)` patterns

Negative / accepted

- No built-in cache invalidation — components re-fetch on mount; accepted for assignment scale
- Large list pages do not paginate on the client — pagination is a server-side API parameter

References

- `web/buildwise-web/src/auth/AuthContext.jsx`
- `web/buildwise-web/src/auth/accessControl.js`
- `web/buildwise-web/src/services/procurementApi.js`
- React Testing Library tests: `src/Features/procurement/components/*.test.jsx`

ADR appendix: ADR-004-flutter-state-management.md

> Scope update (2026-10-05): all seven internal roles now use Flutter. Role-filtered navigation matches React, including procurement and administration. Per-screen StatefulWidget state and injected domain services remain the selected approach; camera/gallery evidence and local notifications are implemented. The older three-role restriction below is historical, not the current behavior.

ADR-004 — Flutter State Management Strategy

Status: Accepted · **Date:** 2026-09-23 · **Owner:** All component owners (Flutter is shared)

Scope: SE3090 spec §8 (Flutter mobile), §9 (device feature, secure storage, API integration)

Context

BuildWise's Flutter application runs on Android (primary submission target: APK) and optionally on Windows desktop and Chrome (web). It must:

1. Authenticate against the BuildWise ASP.NET Core API and store the JWT securely on-device.
2. Navigate role-conditionally to field feature areas. Historical scope: the mobile app was a **field operations tool for three roles** — Site Engineer (material requests), Site Officer (delivery receiving) and Quality Inspector (inspections, NCRs). Procurement, management and administration run on the React web app. Suppliers are external stakeholders contacted by email and have no mobile surface at all.
3. Display live procurement workflow status with **local push notifications** when status changes (device feature — spec §9.2).
4. Support login → authenticated workflow → logout without leaking tokens.

The assignment scale is a team of four working independently on feature folders. Flutter state management must not create hard coupling between feature teams.

Options considered

#	Option	Verdict
A	StatefulWidget + setState per screen, shared AuthService singleton	Chosen — simplest, testable, no extra packages needed
B	Provider / Riverpod	Rejected — adds dependency; team unfamiliar; overkill for four screens
C	BLoC / Cubit	Rejected — heavy boilerplate; difficult to learn quickly mid-sprint
D	GetX	Rejected — opinionated, non-idiomatic, conflicts with Flutter lint rules in <code>analysis_options.yaml</code>
E	InheritedWidget (manual)	Rejected — lower-level than needed; Provider wraps this already

Decision

1. **AuthService** (`lib/core/auth/`) is a singleton holding the JWT, parsed user role, and base API URL. It wraps `flutter_secure_storage` for encrypted on-device token storage. All screens and services read the token from `AuthService`; none store it locally.
2. **Per-screen StatefulWidget + setState** for loading, error, and data state. Each screen owns its own state; no global state store is used.
3. **NotificationService** (`lib/features/procurement/services/notification_service.dart`) wraps `flutter_local_notifications` to display local push notifications when procurement status changes. This is the required **device feature**: on-device notification scheduling without a remote push server, satisfying spec §9.2 without requiring a Firebase project.
4. **ProcurementStatusPoller** (`lib/features/procurement/services/procurement_status_poller.dart`) polls the API on a timer and fires `NotificationService` when status transitions are detected. This is the only background-style process; it runs within the app lifecycle.
5. **Navigation** uses Flutter's built-in `Navigator` with named routes defined in `lib/routes/`. Role-based screen access is enforced in the route resolution logic by reading `AuthService.currentRole`.
6. **No external state management package** beyond the three packages already in `pubspec.yaml` (`http`, `flutter_secure_storage`, `flutter_local_notifications`).

Device feature detail

```

ProcurementStatusPoller (Timer.periodic)

    ↓

GET /api/procurement-workflow/{id}

    ↓

Status changed? —No→ (wait)

    ↓ Yes

NotificationService.showStatusUpdate(title, body)

    ↓

flutter_local_notifications → System notification tray

```

This satisfies the spec requirement for a meaningful device feature (local notifications) without any Firebase dependency, matching the team's decision to avoid a Firebase project for local assignment submission.

Consequences

Positive

- No extra dependencies — `pubspec.yaml` stays minimal and auditable
- `flutter test` works without mocking a state container
- `AuthService` is the single source of truth for the token; no risk of stale tokens in multiple stores
- Notification feature works offline (local, no remote push server)

Negative / accepted

- `setState` causes full-screen rebuilds on data change; acceptable at assignment scale
- Polling instead of WebSocket for status changes; polling interval is configurable
- No persistent offline cache — app requires network for all data; accepted

References

- `mobile/buildwise_mobile/lib/core/auth/`
- `mobile/buildwise_mobile/lib/features/procurement/services/notification_service.dart`
- `mobile/buildwise_mobile/lib/features/procurement/services/procurement_status_poller.dart`
- `mobile/buildwise_mobile/pubspec.yaml`
- Spec §8 (Flutter), §9.2 (device feature)

ADR appendix: ADR-005-deployment-architecture.md

> Deployment status update (2026-10-05): this document records the proposed hosting design, not evidence of a live deployment. No public deployment URLs were verified in the maintenance run. Historical provider free-tier/cost assumptions below must be checked by the team before choosing a current plan. The CI build now takes VITE_API_BASE_URL from a GitHub repository variable, rather than assuming an unverified hosted API URL.

ADR-005 — Deployment Architecture

Status: Accepted · **Date:** 2026-09-23 · **Owner:** Group (all component owners)

Scope: SE3090 spec §11 (deployment), §12 (health, Swagger, public URLs)

Context

The SE3090 spec requires:

- ASP.NET Core API deployed with a public HTTPS URL
- Working /health endpoint
- Working /swagger endpoint
- PostgreSQL deployed securely (credentials via environment variables, not committed)
- React web app deployed with a public HTTPS URL
- Flutter Android APK available for download or sideload
- Agentic AI agent services accessible to the deployed API

Four forces shaped the choice:

1. **Cost** — student project; free tier is a hard requirement.
2. **Simplicity** — no DevOps experience on the team; managed platforms preferred.
3. **PostgreSQL support** — managed Postgres must be available on the same platform.
4. **Environment variable support** — no secrets in code or config files.

Options considered

#	Option	Verdict
A	Railway (API + PostgreSQL + Python services) + Vercel (React)	Chosen — free tier sufficient, managed Postgres included, GitHub-connected deploys
B	Render (API + PostgreSQL) + Netlify (React)	Viable alternative; rejected because Railway's Dockerfile-free deploy is simpler for .NET
C	Azure App Service + Azure Database for PostgreSQL	Rejected — free tier expires after 12 months; credit card required
D	Heroku + Heroku Postgres	Rejected — free dynos removed; costs money
E	AWS Elastic Beanstalk + RDS	Rejected — complex IAM setup; exceeds team's DevOps capacity
F	VPS (e.g., DigitalOcean Droplet)	Rejected — requires manual nginx/SSL setup; maintenance burden

Decision

API — Railway

- ASP.NET Core 8 deployed as a Railway service connected to the GitHub repository.
- `railway.json` (or `Dockerfile` if required) at repository root.
- Environment variables set in Railway dashboard (never in code):
- `ConnectionStrings__DefaultConnection` — Railway-provided PostgreSQL URL
- `Jwt__Key` — signing key
- `AgentService__Url` / `AgentService__RequestUrl` / `AgentService__DeliveryUrl` / `AgentService__QualityUrl` — Railway internal URLs for agent services

PostgreSQL — Railway Managed Postgres

- Railway-provisioned PostgreSQL database.
- EF Core migrations run on startup (`app.MigrateAsync()` in `Program.cs`).
- Credentials injected via `ConnectionStrings__DefaultConnection` environment variable.

Python Agent Services — Railway (4 services)

- Four Railway services (`quotation_agent.py`, `request_agent.py`, `delivery_agent.py`, `quality_agent.py`) each exposed on internal Railway URLs.
- SMTP — configured via `Smtp__Host` / `Smtp__Username` / `Smtp__Password` environment variables; console-logged if `Smtp__Host` is empty.

React — Vercel

- `web/buildwise-web` deployed to Vercel via GitHub integration.
- `VITE_API_BASE_URL` set to the Railway API URL in Vercel environment variables.
- Build command: `npm run build`. Output directory: `dist`.

Flutter APK

- Built locally with `flutter build apk --release`.
- APK (`build/app/outputs/flutter-apk/app-release.apk`) submitted as a binary artifact alongside the repository.
- API base URL in `lib/core/api/api_client.dart` is configurable at build time via a `--dart-define=API_BASE_URL=<url>` flag.

Public URLs (to be populated after deployment)

Service	URL
API	https://buildwise-api.railway.app
Swagger	https://buildwise-api.railway.app/swagger
Health	https://buildwise-api.railway.app/health
React	https://buildwise-web.vercel.app
APK	Submitted as binary with report

Security

- All secrets via environment variables; `.env` is `.gitignored`.
- HTTPS enforced by Railway and Vercel (automatic TLS).
- JWT audience and issuer validated on every request.
- CORS configured to allow only the Vercel origin in production.
- Database credentials never committed.

Consequences

Positive

- Zero infrastructure cost (free tiers)
- GitHub-connected: push to main triggers automatic redeploy
- Managed Postgres: no DBA work, automatic backups
- Vercel CDN: React assets served from edge nodes

Negative / accepted

- Railway free tier has cold-start latency (~5–10 s) after inactivity; acceptable for demonstration
- Python agent services on Railway free tier sleep after 10 min; first agent call after inactivity may timeout (mitigated by the C# in-process fallback in `QuotationAgentClient`)
- APK must be rebuilt and re-submitted manually if the API URL changes

References

- `backend/BuildWise.Api/Program.cs` — `app.MigrateAsync()`, health endpoint registration
- `backend/BuildWise.Api/appsettings.json` — configuration key names (values via env vars)
- `.env.example` — full list of required environment variables

- `web/buildwise-web/package.json` — build scripts
- `mobile/buildwise_mobile/lib/core/api/api_client.dart` — configurable API base URL

ADR appendix: ADR-006-suppliers-are-external-stakeholders.md

Suppliers are external stakeholders — ADR

Status

Accepted. Supersedes the earlier supplier-portal design.

Context

An earlier iteration of BuildWise modelled a Supplier **role**: a login bound to one supplier record through `User.SupplierId`, a signed `supplier_id` JWT claim, a `/api/supplier-portal/*` API surface, a `SupplierPortalController`, a React portal page and a Flutter portal.

That design was rejected because it does not reflect how construction procurement actually works, and it widened the authentication boundary to parties that are not BuildWise employees.

Decision

A supplier is an external stakeholder, not a BuildWise user.

Suppliers do **not**:

- log into BuildWise, or hold an account, JWT, role or password
- have a dashboard, portal or mobile app
- access the BuildWise database directly
- see other suppliers' quotations or any internal procurement data

Communication is external, by email:

```
BuildWise —RFQ email—► External Supplier
External Supplier —quotation email—► Procurement Officer
Procurement Officer —records quotation—► BuildWise
BuildWise —PO email—► External Supplier
External Supplier —physical delivery—► Site Officer (mobile)
BuildWise —NCR / corrective-action email—► External Supplier
```

What is kept — the business entity

Supplier remains a first-class **business entity**, because procurement still

needs the supplier's contact details to send RFQs, record returned quotations, award purchase orders and raise corrective actions. It is referenced by:

- RfqSupplier — which suppliers an RFQ was emailed to
- Quotation — which supplier submitted the recorded quotation
- PurchaseOrder — which supplier the order was placed with
- NonConformance — which supplier a corrective action concerns

POST /api/material-requests/{id}/quotations is the officer-side endpoint that records a quotation **received by email**. It is gated by Policies.SupplierAdministrationOnly (Procurement Officer / Administrator).

Roles

BuildWise has exactly seven internal roles:

```
SiteEngineer → SiteManager → ProcurementOfficer → ProcurementManager
                → SiteOfficer → QualityInspector → Administrator
```

Retired legacy roles

Two earlier aliases were removed once it was verified they were ****permission subsets**** of the roles that replaced them:

Legacy role	Policies it had	Replaced by	Why it was safe
ReceivingOfficer	InternalStaff, DeliveryParticipants	SiteOfficer	SiteOfficer holds both of those plus SiteOperations, QualityReaders and MaterialRequestReaders — a strict superset
ProjectManager	InternalStaff only	SiteManager	SiteManager holds InternalStaff plus both approval policies, procurement reads and delivery — a strict superset

Because each target was a superset, the migration could only ***widen*** access, never narrow it unexpectedly.

The ConsolidateLegacyRoles migration re-points any account still holding a legacy role before deleting the role rows, and de-duplicates so a user already holding the target role is not double-granted. The redundant receiving.officer@buildwise.demo demo login is removed; site.officer@buildwise.demo is the Site Officer account.

The Flutter shell keeps a short-lived legacyAliases map in buildwise_roles.dart purely so a JWT minted ***before*** the migration resolves to the right role instead of locking its holder out. It can be deleted once every client has re-authenticated.

Client split

Client	Roles	Surfaces
React web	all seven	Procurement, approvals, administration
Flutter mobile	Site Engineer, Site Officer, Quality Inspector	Field operations only

Procurement roles signing in on mobile are shown a web-only explainer rather than a partial procurement UI. This keeps purchase orders raisable in exactly one place.

Consequences

Accepted trade-off: a supplier cannot self-serve a quotation. Every quotation is transcribed by the Procurement Officer. That is manual work, but it is the real process, and it keeps authentication limited to internal staff.

Enforced by tests: a token carrying a legacy Supplier role is refused by every endpoint (`RbacAuthorizationTests.cs`), the role is not seeded, and no user row is bound to a supplier.

Migration

Two migrations were applied to the development database:

20261002104811_RemoveSupplierUserAccount — deletes legacy supplier login rows and the Supplier role, then drops the `users.SupplierId` foreign key. It **never** touches the `suppliers` table, so all supplier business records and every RFQ / quotation / purchase-order relationship are preserved.

20261002113029_ConsolidateLegacyRoles — re-points accounts holding a retired role at their canonical replacement (de-duplicated), deletes the now-unused demo login, and removes the ProjectManager, ReceivingOfficer and Supplier role rows.

Verified after applying: roles holds exactly seven rows, no `users.SupplierId` column remains, and supplier / RFQ / quotation / purchase-order counts are intact.

Measured performance appendix

```
{  
  
  "measuredAtUtc": "2026-10-05T09:30:52.975Z",  
  
  "environment": "Local Windows development stack; not a production SLA",  
  
  "requestsPerScenario": 30,  
  
  "concurrencyPerScenario": 5,  
  
  "results": [  
  
    {  
  
      "name": "Shared ASP.NET Core API + PostgreSQL",  
  
      "total": 30,  
  
      "concurrency": 5,  
  
      "succeeded": 30,  
  
      "failed": 0,  
  
      "successRatePct": 100,  
  
      "throughputRps": 267.7,  
  
      "latencyMs": {  
  
        "average": 17.59,  
  
        "p50": 12.86,  
  
        "p95": 78.89,  
  
        "max": 79.83  
  
      },  
  
      "localAcceptanceP95Ms": 2000,  
  
      "passed": true,  
  
      "sampleErrors": []  
  
    },  
  
    {  
  
      "name": "Agent 8001 quotation analysis",  
  
      "total": 30,  
  
      "concurrency": 5,  
  
      "succeeded": 30,  
  
      "failed": 0,  
  

```

```

    "successRatePct": 100,

    "throughputRps": 2.98,

    "latencyMs": {

        "average": 1625.93,

        "p50": 817.64,

        "p95": 6782.05,

        "max": 8522.86

    },

    "localAcceptanceP95Ms": 1000,

    "passed": false,

    "sampleErrors": []

},

{

    "name": "Agent 8002 request analysis",

    "total": 30,

    "concurrency": 5,

    "succeeded": 30,

    "failed": 0,

    "successRatePct": 100,

    "throughputRps": 223.05,

    "latencyMs": {

        "average": 21.93,

        "p50": 8.07,

        "p95": 94.49,

        "max": 94.63

    },

    "localAcceptanceP95Ms": 1000,

    "passed": true,

    "sampleErrors": []

},

{

    "name": "Agent 8003 discrepancy analysis",

```

```

    "total": 30,

    "concurrency": 5,

    "succeeded": 30,

    "failed": 0,

    "successRatePct": 100,

    "throughputRps": 256.81,

    "latencyMs": {

        "average": 19.12,

        "p50": 6.9,

        "p95": 82.03,

        "max": 82.29

    },

    "localAcceptanceP95Ms": 1000,

    "passed": true,

    "sampleErrors": []

},

{

    "name": "Agent 8004 quality-risk analysis",

    "total": 30,

    "concurrency": 5,

    "succeeded": 30,

    "failed": 0,

    "successRatePct": 100,

    "throughputRps": 291.31,

    "latencyMs": {

        "average": 16.75,

        "p50": 9.11,

        "p95": 58.02,

        "max": 58.59

    },

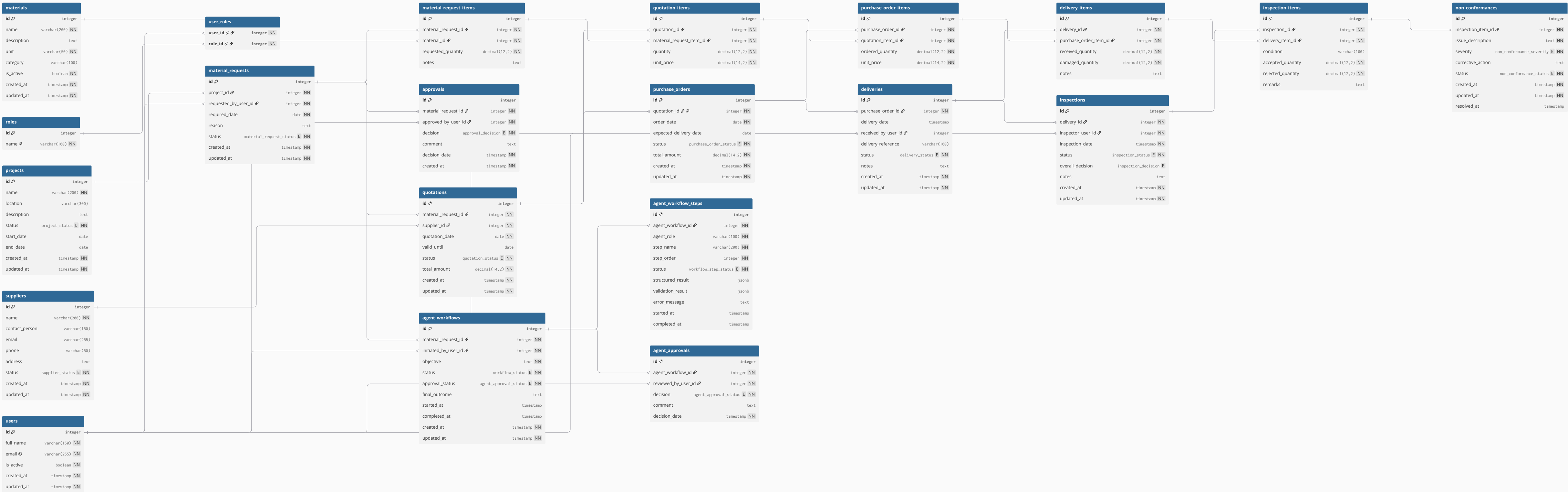
    "localAcceptanceP95Ms": 1000,

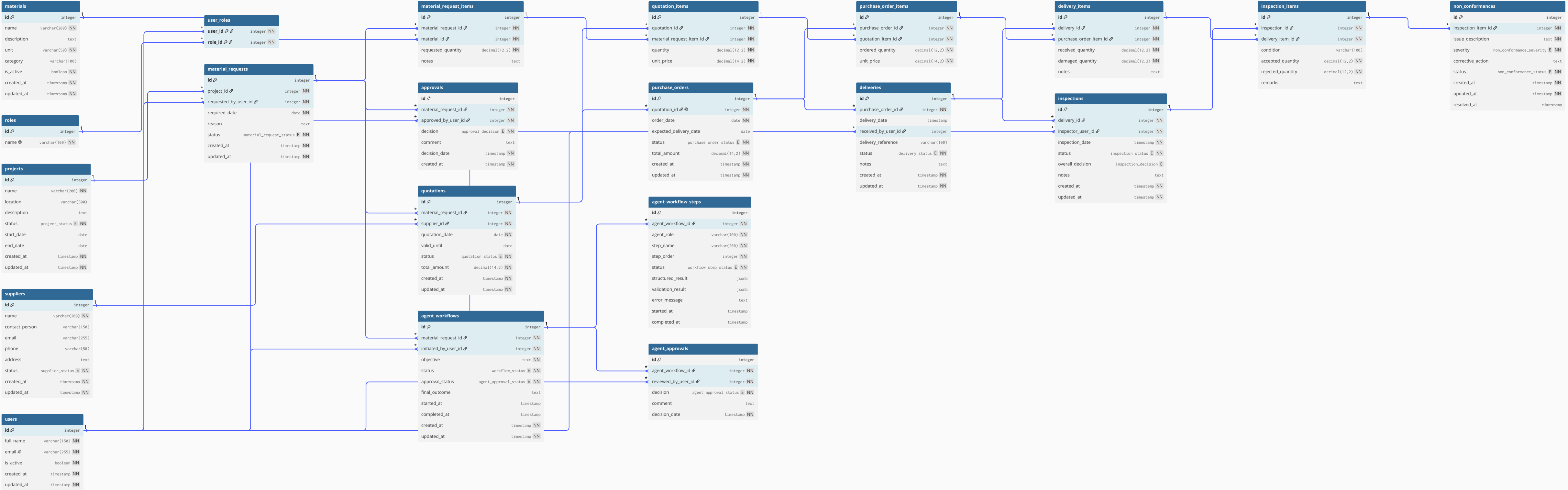
    "passed": true,

    "sampleErrors": []

```

```
}  
]  
}
```





```
Enum project_status {
    Planned
    Active
    Completed
    Cancelled
}
```

```
Enum material_request_status {
    Draft
    PendingApproval
    Approved
    Rejected
}
```

```
Enum approval_decision {
    Approved
    Rejected
    RevisionRequested
}
```

```
Enum supplier_status {
    Active
    Inactive
    Suspended
}
```

```
Enum quotation_status {
    Submitted
    UnderReview
    Selected
    Rejected
    Expired
}
```

```
Enum purchase_order_status {
    Created
    Confirmed
    InProgress
    Completed
    Cancelled
}
```

```
Enum delivery_status {
    Scheduled
    InTransit
    Delivered
    Received
    DiscrepancyReported
}
```

```
Enum inspection_status {
    Pending
    UnderInspection
    Completed
}
```

```
Enum inspection_decision {
    Accepted
    PartiallyAccepted
    Rejected
}
```

```
Enum non_conformance_severity {
    Low
    Medium
    High
    Critical
}
```

```
Enum non_conformance_status {
    Open
    CorrectiveActionRequired
    Resolved
    Closed
}
```

```
Enum workflow_status {
    Pending
    Running
    AwaitingApproval
    Completed
    Failed
    Cancelled
}
```

```
Enum workflow_step_status {
    Pending
    Running
    Completed
    Failed
    Skipped
}
```

```
Enum agent_approval_status {
    Pending
    Approved
    Rejected
    RevisionRequested
}
```

SE3090 ASSIGNMENT 1 | BUILDWISE PROJECT SCENARIO

Construction Materials Procurement, Delivery and Quality Management System

BuildWise - Integrated Full-Stack and Agentic AI Application

TEAM REFERENCE

This document explains the complete BuildWise scenario in one place so every team member can understand the problem, users, four components, shared workflow, four AI contributions, human approvals, web/mobile responsibilities and core data.

Real-world problem

Construction material procurement becomes difficult when material needs start at a construction site but supplier selection and purchasing decisions happen in an office. Requests, quotations, approvals, purchase orders, deliveries and inspections may be tracked through separate calls, messages, spreadsheets and paper records.

BuildWise connects those activities in one system. Site users use a Flutter mobile application for material requests, delivery receiving and quality inspection. Procurement and management staff use a React web application for supplier and quotation management, approvals, purchase orders, monitoring and administration. ASP.NET Core provides the shared secure API and business rules, PostgreSQL stores the shared relational data, and Agentic AI assists decisions while important actions remain under human control.

User roles

- 1. Site Engineer / Site Officer.** Creates material requests, tracks progress and records deliveries from the construction site.
- 2. Procurement Officer.** Maintains suppliers, records quotations, compares options and prepares procurement.
- 3. Procurement Manager.** Reviews request approvals and AI-assisted procurement recommendations, then approves, rejects or requests revision.
- 4. Quality Inspector.** Inspects delivered materials, records accepted/rejected quantities and manages non-conformance cases.
- 5. Administrator.** Manages users, roles and shared system settings.

Implementation note. User authentication and role-based authorization are shared mandatory features and should not be counted as one of the four main business components. Supplier records are business data managed by procurement staff.

Four main business components

Component	Main functionality	Business-specific operation
Material Request & Approval Management	Projects, materials, requests, request items, required dates, reasons, statuses and request approval decisions.	Submit and control a material request, preserve approval history and prevent silent changes after approval.
Supplier, Quotation & Procurement Management	Suppliers, quotation items, validity, comparison, AI-assisted recommendation, procurement approval, purchase orders and purchase-order items.	Evaluate eligible quotations and create a purchase order only after the recommendation passes validation and authorized approval.
Delivery & Material Receiving Management	Expected purchase-order information, deliveries, received quantities, damaged quantities and delivery status.	Reconcile what was ordered against what actually arrived and flag shortages, damage or partial delivery.
Quality Inspection & Non-Conformance Management	Inspections, inspection items, accepted/rejected quantities, quality decisions and non-conformance records.	Complete the inspection and create a traceable non-conformance when rejected material requires corrective action.

Each student owns one component and should implement its ASP.NET Core endpoints, PostgreSQL entities and relationships, React interfaces, Flutter interfaces, tests, documentation, Git evidence and one identifiable Agentic AI contribution.

Example complete scenario

A Site Engineer submits this objective using Flutter:

“Request 250 bags of OPC cement for the Colombo apartment project, required by 22 August for ground-floor column concreting.”

The request is reviewed and approved for procurement. The Procurement Officer then records three supplier quotations:

- **Supplier A:** active, full 250 bags available, valid quotation, total LKR 525,000.
- **Supplier B:** suspended, full 250 bags available, lower total of LKR 510,000.
- **Supplier C:** active and cheapest, but only 200 bags are quoted.

Workflow

1. Flutter submission. The Site Engineer logs in, selects the project and material, enters 250 bags, the required date and reason, and submits the request.

2. ASP.NET Core processing. The API validates the JWT, role, project, material, quantity and date, then saves the request and items in PostgreSQL.

3. Request Validation & Planning Agent. The component-owned agent checks completeness, quantities, dates and urgency, then produces a structured request summary and planning flags. It does not approve the request.

4. Request approval. An authorized manager reviews the request and records Approve, Reject or Revision Requested. Approved requests may continue to procurement.

Workflow continued

5. React procurement preparation. The Procurement Officer records supplier quotations. Each quotation is linked to the original material request, and each quotation item is linked to the corresponding request item.

6. Quotation & Supplier Analysis Agent. This component-owned agent compares eligible quotations using structured data such as supplier status, quotation validity, quantities and calculated totals. It identifies that Supplier B is suspended and Supplier C does not cover the full quantity.

7. Structured recommendation. The agent recommends Supplier A and returns a short structured rationale/evidence summary, ranked alternatives and warnings. BuildWise stores observable summaries and results - not private model chain-of-thought.

Deterministic validation

Before the recommendation can reach procurement approval, ASP.NET Core verifies fixed rules:

- the material request exists and has the required approval;
- the recommended supplier exists and has Active status;
- the recommended quotation exists and is not expired;
- quotation items belong to the requested material items;
- the recommended quantity covers the requested quantity;
- the quotation total matches the quotation-item calculation;
- the recommendation follows the required structured output schema;
- purchase-order creation is blocked until authorized procurement approval exists.

Human approval

8. Procurement Manager review. The workflow becomes **AwaitingApproval**. The Procurement Manager uses React to review the recommended supplier, structured rationale, warnings, validation results and execution summary.

The Procurement Manager can:

- **Approve**
- **Reject**
- **Request Revision**

9. Final procurement execution. If approved, ASP.NET Core records the agent workflow approval and creates the purchase order and purchase-order items from the selected quotation. Purchase orders remain owned by Component 2; Component 3 only consumes approved purchase orders for receiving.

10. Flutter status update. The Site Engineer receives the approved procurement status through the mobile application.

Therefore, the assessed procurement workflow is:

Flutter → ASP.NET Core → PostgreSQL → Agentic AI → React approval → ASP.NET Core / PostgreSQL → Flutter status update

Delivery and quality continuation

BuildWise continues the same traceable procurement record after purchase-order approval:

11. Delivery receiving. When the supplier arrives, the Site Officer opens the approved purchase order in Flutter and records the actual delivery. Example: 250 bags ordered, 240 bags received and 5 of the received bags visibly damaged.

12. Delivery Discrepancy Agent. This component-owned agent compares ordered, received and damaged quantities. It produces a structured discrepancy summary such as shortage = 10 bags and damaged = 5 bags, with warnings for partial or abnormal delivery. The backend independently validates the arithmetic before saving the result.

13. Quality inspection. The Quality Inspector opens the delivery in Flutter, performs the inspection and records accepted/rejected quantities. Example: 235 bags accepted and 5 bags rejected.

14. Quality Risk & Non-Conformance Agent. This component-owned agent analyses the inspection result and existing supplier-quality evidence, then suggests risk flags and an appropriate non-conformance/corrective-action summary when required. The Quality Inspector remains responsible for the recorded inspection decision.

15. Non-conformance. If rejected material requires corrective action, BuildWise creates a non-conformance record linked to the affected inspection item and tracks it until resolved or closed.

Complete BuildWise business flow

Material Request → Request Approval → Quotations → AI-Assisted Procurement Evaluation → Manager Approval → Purchase Order → Delivery → AI Discrepancy Analysis → Quality Inspection → AI Quality-Risk Assistance → Acceptance / Non-Conformance

Four specialized Agentic AI contributions

Owned component	Agent	Responsibility
Component 1 - Material Request & Approval	Request Validation & Planning Agent	Checks completeness, quantities, dates and urgency; creates a structured request plan/flags.
Component 2 - Supplier, Quotation & Procurement	Quotation & Supplier Analysis Agent	Compares eligible quotations and supplier evidence; returns ranked options, structured rationale and warnings.
Component 3 - Delivery & Receiving	Delivery Discrepancy Agent	Compares ordered vs received/damaged quantities and flags shortages, partial deliveries or discrepancies.
Component 4 - Quality Inspection & NCR	Quality Risk & Non-Conformance Agent	Assists with inspection-risk interpretation and prepares structured NCR/corrective-action suggestions.

Shared safety rule: deterministic backend validation is a system-wide control, not a fifth AI agent. Agents may analyse and recommend, but authorization, calculations, state transitions and other high-impact actions are enforced by ASP.NET Core and authorized users.

React web application

The React application is mainly for procurement, management, administration and monitoring:

- material-request review and approval visibility;
- supplier management and supplier status control;
- quotation entry and side-by-side comparison;
- Agentic AI workflow start, status, structured summaries and execution-history views;
- recommended supplier, ranked alternatives, structured rationale/evidence summary, warnings and validation results;
- Procurement Manager Approve / Reject / Request Revision controls;
- purchase-order creation and management;
- delivery discrepancy and quality/non-conformance history;
- user and role administration;
- search, filtering, sorting, pagination, dashboards and reports.

Flutter mobile application

The Flutter application is mainly for construction-site operations:

- registration/login/logout and secure JWT token storage;
- create material requests and select required dates;
- track request and procurement status;
- view approved purchase-order information required for site receiving;
- record delivery quantities, damaged quantities and delivery references;
- perform quality inspections and record accepted/rejected quantities;
- view non-conformance status where relevant;
- receive push notifications for important workflow changes.

Third-party integration

Firebase Cloud Messaging (FCM) can be integrated through ASP.NET Core to send mobile notifications such as request approval, procurement recommendation awaiting manager approval, purchase-order creation, delivery discrepancy and inspection completion/rejection. Firebase credentials must remain server-side and failures must be handled safely.

Core data used by BuildWise

Area	Main entities
Identity	users, roles, user_roles
Material Request	projects, materials, material_requests, material_request_items, approvals
Procurement	suppliers, quotations, quotation_items, purchase_orders, purchase_order_items
Delivery	deliveries, delivery_items
Quality	inspections, inspection_items, non_conformances
Agentic AI	agent_workflows, agent_workflow_steps, agent_approvals